Implement the PUT Endpoint for Updating an Existing Product

Explore how to implement a PUT method endpoint in Flask to update an existing product by its ID. Understand the use of route decorators, request JSON data handling, and returning the updated product as a JSON response with a 200 OK status.

REST API uses PUT to update an existing resource in a collection. We'll see how to implement an API endpoint that uses PUT to update an existing product in the collection.

The `request` object of Flask can be used to retrieve the body of a PUT request, and path parameter mapping can be used to get the ID of the product to be updated. The `Route` decorator can be used to map a method to the PUT request. Let's see how to do the same.

Steps

1. Open `server/api/products_api.py`.
2. Add a new method. Name the method `update_product`. Provide it the required parameter, which is product ID, and name the parameter `product_id`.

```
def update_product(product_id):
```

Ask

3. Add the following code.

```
    data = request.get_json()
    updated_product = None
    for product in products:
        if product['id'] == product_id:
            product.update(data)
            updated_product = product
    return jsonify(updated_product)
```

4. Decorate the method with the `route` decorator of `products_blueprint`. Pass the URI that will represent the endpoint and the supported HTTP method, which is PUT.

```
@products_blueprint.route('/products/<product_id>', methods=['PUT'])
```

5. Run the application.
6. Hit the following URL using your preferred tool. When testing on the Educative platform, please replace `http://localhost` with the URL provided in the SPA widget.

```
$ curl -X PUT --data '{"version": "3.0"}' -H'Content-Type: application/json'
```

7. This should give the following output.

```
< HTTP/1.1 200 OK
< Content-Type: application/json
< Content-Length: 84
{"description":"container for rest","id":"3","name":"rest-container","versio
```

How it works

For the PUT request mapping, the path parameter of the URI is mapped to the route parameter of the `route` decorator. This, in turn, gets passed as the method parameter. In our case, the path parameter of the URI is the `product_id`.

Ask

Request/response for PUT API for updating an existing product

The `request` object has a `get_json` method that returns the JSON passed as the body of the PUT request. We've assigned the JSON object to the `data` variable.

Next, using the ID of the product (`product_id`) and the data we received as body (`data`), we update the corresponding product in the product list. Then, the updated product is returned as the response. Since we're not performing a long-running operation, the response can have a status of `200 OK`. Hence, we send back `200 OK` as the status for the response.

Ask

Try it yourself

- The widget below shows the code created during the previous lesson.

- Follow the steps above to complete the code for this lesson.
- Click “Run” in the widget below to start the service. The app URL for accessing the service is mentioned at the bottom of the widget.
- Wait for the service to start before accessing the URL.
- Append `/3` to the app URL and provide the JSON body to send the PUT request. Set the `Content-Type` header as `application/json`.

> **Note:** Use REST API clients, such as Postman and Insomnia, or command line tools like curl to send the request.

- Send a GET request by appending `/3` to the app URL to verify that the update was successful.

> **Note:** Access the URL using any web browser, REST API clients, such as Postman and Insomnia, or command line tools like curl.

Ask

Run the code above, and access the URL once the service starts, to view the output

Solution

If stuck, refer to the final code below for help.

Please note that the file below is read-only. The changes from the previous lesson are highlighted.

products_api.py Python 3.8

```
from flask import (request, jsonify, Blueprint, make_response, url_for)
from .data_store import products

products_blueprint = Blueprint('products_api', __name__, url_prefix='/api


@products_blueprint.route('/products', methods=['GET'])
def list_products():
    return jsonify(products)


@products_blueprint.route('/products/<product_id>', methods=['GET'
def get_product(product_id):
```

Ask

Solution code for this lesson

Ask